

INDEX

Name : DIYOSH KUNAR Class : _____
 Section : 9-D Roll No. : _____ Subject : _____

Sl. No.	Date	Title	Page No.	Teacher's Sign. / Remarks
1)	27/02	WEEK 0 :- P1 to P6	-	
2)	6/03	WEEK 1 :- FCFS SJF → Non-Pri ↳ Pre-emptive	-	A 13/3
3)	13/3	WEEK 2 :- Priority Scheduling → Non-Pri ↳ Pre-emptive	-	
4)	27/3	WEEK 3 :- Round Robin Multi-Level Queue	-	A 27/3
5)	10/4	Week 4 :- Rate Monotonic Sched Earliest Deadline First Proportional Share Scheduling	-	A 5/4/26
6)	24/4	Week 5 :- Bounded Buffer Dining Philosopher	-	A 24/4
7)	8/5	Week 6 :- Banker's Algo Deadlock Def ⁿ	<u>10</u>	A 8/5/26
8)	15/5	Week 7 :- Contiguous Memory Allocation	-	9/10/26
9)	22/5	Week-8 :-	-	A 22/5

P①

WAP to find second smallest element in array

```
#include <stdio.h>
int arr[7];
void main() {
    int a, s1, s2;
    printf("Enter array elements: \n");
    for (int i = 0; i < 7; i++) {
        scanf("%d", &a);
        arr[i] = a;
    }
    s1 = arr[0];
    for (int k = 0; k < 7; k++) {
        if (arr[k] == s1) {
            continue;
        }
        else if (arr[k] < s2) {
            s2 = arr[k];
        }
    }
    printf("2nd smallest element: ");
    printf("%d", s2);
}
```

dp Enter array elements
2 4 9 8 7 6
2nd smallest element: 2

(P2) WAP to find sum of left diagonal

```
#include <stdio.h>
#include <stdlib.h>
```

```
int arr[3][3];
```

```
void main()
```

```
{
    int a, i, j;
```

```
printf("Enter matrix elements");
```

```
for (int x=0; x<3; x++)
```

```
{
    for (int y=0; y<3; y++)
```

```
{
    scanf("%d", &a);
```

```
arr[x][y]=a;
```

```
}
```

```
}
```

```
printf("Matrix: \n");
```

```
for (int x=0; x<3; x++)
```

```
{
    for (int y=0; y<3; y++)
```

```
{
    printf("%d", arr[x][y]);
```

```
}
```

```
printf("\n");
```

```
}
```

```
printf("Sum of left diagonal: ");
```

```
i = (arr[0][0] + arr[1][1] + arr[2][2]);
```

```
printf("%d", i);
```

```
}
```

~~o/p~~ Enter matrix elements: 1 2 3 4 5 6 7 8 9

Matrix

1 2 3

4 5 6

7 8 9

Sum of left diagonal: 15

(P3) WAP to find sum of rows & columns

```
#include <stdio.h>
```

```
void main()
```

```
{
    int rows, cols, i, j, sum;
```

```
printf("Enter num of rows and cols: ");
```

```
scanf("%d %d", &rows, &cols);
```

```
int Matrix[rows][cols];
```

```
printf("Enter elements \n");
```

```
for (i=0; i<rows; i++)
```

```
{
    for (j=0; j<cols; j++)
```

```
{
    scanf("%d", &Matrix[i][j]);
```

```
}
```

```
}
```

```
printf("\n Row Sums \n");
```

```
for (i=0; i<rows; i++)
```

```
{
    sum = 0;
```

```
for (j=0; j<cols; j++)
```

```
{
    sum += Matrix[i][j];
```

```
}
```

```
printf("Sum of Row %d = %d \n", i+1, sum);
```

```
}
```

```
printf("\n Column Sums \n");
```

```
for (j=0; j<cols; j++)
```

```
{
    sum = 0;
```

```
for (i=0; i<rows; i++)
```

```
{
    sum += Matrix[i][j];
```

```
}
```

```
printf("Sum of Column %d = %d \n", j+1, sum);
```

```
}
```

```
return 0;
```

```
}
```

~~o/p~~ Enter elements of matrix:

1
2
3
4
5
6
7
8
9

Row Sums:

Sum of Row 1 = 6

Sum of Row 2 = 15

Sum of Row 3 = 24

Column Sums:

Sum of Col 1 = 12

Sum of Col 2 = 15

Sum of Col 3 = 18

(P4)

WAP to Count total numbers of duplicate elements

```

int main() {
    int arr[100];
    int size, i, j, count = 0;
    printf("Enter size of array: ");
    scanf("%d", &size);
    printf("Enter elements:");
    for (i = 0; i < size; i++) {
        scanf("%d", &arr[i]);
    }
    for (i = 0; i < size; i++) {
        for (j = i + 1; j < size; j++) {
            if (arr[i] == arr[j]) {
                count++;
                break;
            }
        }
    }
    printf("\n total no. of Duplicates = %d", count);
    return 0;
}

```

0/p Enter size of array: 5
Enter elements: 1 2 2 3 4

Total no. of Duplicates: 2

(P5)

WAP to find second largest element

```

#include <stdio.h>
#include <stdlib.h>
int arr[7];
void main() {
    int a, l1, l2;
    printf("Enter array elements: ");
    for (int j = 0; j < 7; j++) {
        if (arr[j] > l1) {
            l1 = arr[j];
        }
    }
    l2 = arr[0];
    for (int k = 0; k < 7; k++) {
        if (arr[k] == l1) {
            continue;
        }
        else if (arr[k] > l2) {
            l2 = arr[k];
        }
    }
    printf("2nd largest element: ");
    printf("%d", l2);
}

```

0/p Enter array elements: 1 2 3 4 8 9 6

2nd largest element: 8

Q6) WAP to delete an element at desired position from array.

```
#include <stdio.h>
```

```
void main()
```

```
{ int size, pos, j;
```

```
printf("Enter size of array:");
```

```
scanf("%d", &size);
```

```
printf("Enter elements in array:");
```

```
for(int i=0; i<size; i++)
```

```
{ scanf("%d", &arr[i]);
```

```
} printf("Enter the pos to be deleted:");
```

```
scanf("%d", &pos);
```

```
if(pos > size) { printf("Invalid pos"); }
```

```
else { for(j=0; j<pos-1; j++)
```

```
{ while(j<size)
```

```
{ arr[j] = arr[j+1];
```

```
j = j+1;
```

```
} printf("Array: \n");
```

```
for(int i=0; i<size-1; i++)
```

```
{ printf("%d", arr[i]);
```

```
} // Enter size of array: 5
```

```
// Enter elements in array: 1 2 3 4 5
```

```
// Enter pos to be deleted: 4
```

```
Array:- 1 2 3 5
```

Week 1.1

Date _____
Page _____

FCFS

```
#include <stdio.h>
```

```
{ int n, c, sum, sum2, sum1;
```

```
printf("Enter number of processes");
```

```
scanf("%d", &n);
```

```
int at[n], bt[n], ct[n], tat[n], wt[n], rt[n];
```

```
float avg_tat, avg_wt, avg_rt;
```

```
sum1 = sum2 = sum3 = 0;
```

```
for(i=0; i<n; i++)
```

```
{ printf("Enter Arrival Time and Burst
```

```
Time for P %d: ", i+1);
```

```
scanf("%d %d", &at[i], &bt[i]);
```

```
} ct[0] = at[0] + bt[0];
```

```
for(i=1; i<n; i++)
```

```
{ if(ct[i-1] < at[i])
```

```
ct[i] = at[i] + bt[i];
```

```
else
```

```
ct[i] = ct[i-1] + bt[i];
```

```
} for(c=0; c<n; c++)
```

```
{ tat[c] = ct[c] - at[c];
```

```
wt[c] = tat[c] - bt[c];
```

```
rt[c] = wt[c];
```

```
} for(i=0; i<n; i++)
```

```
{ sum1 += tat[i];
```

```
sum2 += wt[i];
```

```
sum3 += rt[i];
```

```
}
```



```

avgtat = (float) sum1 / n;
avgwt = (float) sum2 / n;
avgrt = (float) sum3 / n;
printf("\n P1: AT: 0 BT: 2 CT: 2 TAT: 2 WT: 0 RT: 0\n");
for(i=0; i<n; i++) {
    printf("P %d: AT: %d BT: %d CT: %d TAT: %d WT: %d RT: %d\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);
}
printf("\n Average. TAT: %.2f | n Average. WT: %.2f | n Average. RT: %.2f | n", avgtat, avgwt,
    avgrt);

```

return 0;

}

o/p: Enter number of processes: 4
 Enter Arrival time and BT for P1: 0 2
 Enter AT and BT for P2: 1 2
 Enter AT and BT for P3: 5 3
 Enter AT and BT for P4: 6 4

P	AT	BT	CT	TAT	WT	RT
P ₁	0	2	2	2	0	0
P ₂	1	2	4	3	1	1
P ₃	5	3	8	3	8	0
P ₄	6	4	12	6	2	2

Avg TAT: 3.50
 Avg WT: 0.75
 Avg RT: 0.75

a

SJF - non-preemptive

```

#include <stdio.h>
int main() {
    int n, i, time = 0, completed = 0, shortest;
    printf("Enter number of processes:");
    scanf("%d", &n);
    int at[n], bt[n], ct[n], tat[n], wt[n], rt[n];
    int visited[n];
    int sumtat = 0, sumwt = 0, sumrt = 0;
    float avgtat, avgwt, avgrt;

    for(i=0; i<n; i++) {
        printf("Enter Arrival time & burst time for P %d:", i+1);
        scanf("%d %d", &at[i], &bt[i]);
        visited[i] = 0;
    }

    while(completed < n) {
        shortest = -1;
        for(i=0; i<n; i++) {
            if(at[i] <= time && visited[i] == 0) {
                if(shortest == -1 || bt[i] < bt[shortest]) {
                    shortest = i;
                }
            }
        }

        if(shortest == -1) {
            time++;
            continue;
        }

        time = bt[shortest];
        time += bt[shortest];
    }
}

```



```

if (p[i].ct[shortest] == time;
    visited[shortest] = 1;
    Completed++;

```

```

    tat[shortest] = ct[shortest] + at[shortest];
    wt[shortest] = tat[shortest] - bt[shortest];
    rt[shortest] = wt[shortest];

```

```

    sumtat += tat[shortest];
    sumwt += wt[shortest];
    sumrt += rt[shortest];

```

```

}
avgtat = (float) sumtat / n;
avgwt = (float) sumwt / n;
avgrt = (float) sumrt / n;

```

```

printf("\n P | t AT | t BT | t CT | t TAT | t WT | t RT | \n");

```

```

for (i = 0; i < n; i++) {

```

```

    printf("P %d | t %d | t %d | t %d | t %d | t %d | t %d | \n",
           i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);

```

```

}
printf("\n Average TAT: %.2f, avg tat);
printf("\n Average WT: %.2f, avg wt);
printf("\n Average RT: %.2f, avg rt);

```

```

return 0;
}

```

#

Enter number of processes: 3
 Enter arrival time and burst time for P1: 0 2
 Enter arrival time and burst time for P2: 1 2
 Enter arrival time and burst time for P3: 5 3

P	AT	BT	CT	TAT	WT	RT
P1	0	2	2	2	0	0
P2	1	2	4	3	1	1
P3	5	3	8	3	0	0

Average TAT: 2.67

Average WT: 0.33

Average RT: 0.33

SJF preemptive

```
#include <stdio.h>
int main() {
    int n, i, time = 0, smallest count = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int at[n], bt[n], ct[n], tat[n], wt[n], rt[n];
    start[n];
```

```
float avg_tat, avg_wt, avg_rt;
```

```
for (i = 0; i < n; i++) {
    printf("Enter Arrival Time & Burst Time  
for p %d:", i+1);
    scanf("%d %d", &at[i], &bt[i]);
    rt[i] = bt[i];
    start[i] = -1;
```

```
}
while (count != n) {
    smallest = -1;
    for (i = 0; i < n; i++) {
```

```
        if (at[i] <= time & rt[i] > 0) {
            if (smallest == -1 || rt[i] < rt[smallest])
                smallest = i;
        }
```

```
    }
    if (smallest == -1) {
        time++;
        continue;
    }
```

```
    if (start[smallest] == -1) {
        start[smallest] = time;
        rt[smallest]--;
        time++;
    }
```

```
    if (rt[smallest] == 0) {
        count++;
        ct[smallest] = time;
```

```
    }
    for (i = 0; i < n; i++) {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
        rt = time - start[i] - at[i];
```

```
    sum_tat += tat[i];
    sum_wt += wt[i];
    sum_rt += rt[i];
```

```
    avg_tat = (float) sum_tat / n;
    avg_wt = (float) sum_wt / n;
    avg_rt = (float) sum_rt / n;
```

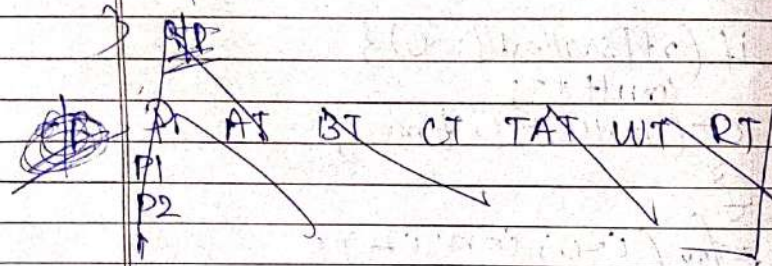
```
    printf("\n P | t AT | t BT | t CT | t TAT | t  
wt | t RT | n);
```

```
    for (i = 0; i < n; i++) {
        printf("P %d | t %d | t %d | t %d | t %d  
| t %d | t %d | n %d | i+1, at[i],  
bt[i], ct[i], tat[i], wt[i], start[i] - at[i];
```



```
printf("\n Average TAT: %.2f, avg tat);
printf("\n Average WT: %.2f, avg wt);
printf("\n Average RT: %.2f, avg rt);
```

```
return 0;
```



Enter number of processes: 3
 Enter AT & BT for P1: 0 2
 Enter AT & BT for P2: 1 2
 Enter AT & BT for P3: 5 3

P	AT	BT	CT	TAT	WT	RT
P1	0	2	2	2	0	0
P2	1	2	4	3	1	1
P3	5	3	8	3	0	0

Average TAT: 2.67

Average WT: 0.33

Average RT: 0.33

Week-2

Date _____
Page _____

Priority Scheduling (Non-Preemptive)

```
#include <stdio.h>
```

```
struct process {
```

```
int pid;
```

```
int at;
```

```
int bt;
```

```
int pr;
```

```
int ct;
```

```
int tat;
```

```
int rt;
```

```
int visited;
```

```
};
```

```
int main() {
```

```
int n, time = 0, completed = 0, i, highest;
```

```
printf("Enter number of processes: ");
```

```
scanf("%d", &n);
```

```
struct process p[n];
```

```
for (i = 0; i < n; i++) {
```

```
    p[i].pid = i + 1;
```

```
    printf("Enter AT, BT and priority for P%d:",
```

```
        p[i].pid);
```

```
    scanf("%d %d %d", &p[i].at, &p[i].bt,
```

```
        &p[i].pr);
```

```
    p[i].visited = 0;
```

```
}
```

```
int sumtat = 0, sumat = 0, sumrt = 0;
```

```
float avgat, avgwt, avgrt;
```

```
while (completed < n) {
```



```

highest = -1;
for (i=0; i<n; i++) {
    if (p[i].at < time & p[i].visited == 0) {
        if (highest == -1 || p[i].pr < p[highest].pr) {
            highest = i;
        }
    }
}

if (highest == -1) {
    time++;
    continue;
}

p[highest].rt = time - p[highest].at;
time += p[highest].bt;
p[highest].ct = time;
p[highest].visited = 1;
completed++;

p[highest].tat = p[highest].ct - p[highest].at;
p[highest].wt = p[highest].tat - p[highest].rt;

sumtat += p[highest].tat;
sumwt += p[highest].wt;
sumrt += p[highest].rt;

avgtat = (float) sumtat / n;
avgwt = (float) sumwt / n;
avgrt = (float) sumrt / n;

printf("\nP | AT | BT | PT | CT | TAT | WT | RT | \n");

for (i=0; i<n; i++) {

```

```

printf("P %d | AT %d | BT %d | PT %d | CT %d | TAT %d | WT %d | RT %d | \n",
       p[i].pid,
       p[i].at, p[i].bt, p[i].pr, p[i].ct, p[i].tat, p[i].wt, p[i].rt);
}

```

```

printf("\nAverage TAT: %.2f, avgwt: %.2f, avgrt: %.2f, \n");
return 0;
}

```

op Enter number of processes: 3
 Enter AT, BT and Priority for P1: 1 5 3
 Enter AT, BT and Priority for P2: 4 2 1
 Enter AT, BT and Priority for P3: 0 3 2

P	AT	BT	PR	CT	TAT	WT	RT
P1	1	5	3	8	7	2	2
P2	4	2	1	10	6	4	4
P3	0	3	2	3	3	0	0

Average TAT: 5.33
 Average WT: 2.00
 Average RT: 2.00

Preemptive

```
#include <stdio.h>

struct process {
    int pid, at, bt, pr, ct, tat, wt, rt;
    int remaining;
    int visited;
};

int main() {
    int n, time = 0, completed = 0, i, highest;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    struct process p[n];
    for (i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("Enter at, bt and priority for P%d:", p[i].pid);
        scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].pr);
        p[i].remaining = p[i].bt;
        p[i].visited = 0;
    }

    int sumtat = 0, sumwt = 0, sumrt = 0;
    float avgat, avgwt, avgrt;

    while (completed < n) {
```

AKshay Katter
intel

Date: _____
Page: _____

```
highest = -1;
for (i = 0; i < n; i++) {
    if (p[i].at <= time & &p[i].remaining > 0) {
        if (highest == -1 || p[i].pr < p[highest].pr)
            highest = i;
    }
}

if (highest == -1) {
    time++;
    continue;
}

if (p[highest].visited == 0) {
    p[highest].rt = time - p[highest].at;
    p[highest].visited = 1;
}

p[highest].remaining--;
time++;

if (p[highest].remaining == 0) {
    p[highest].ct = time;
    completed++;

    p[highest].tat = p[highest].ct - p[highest].at;
    p[highest].wt = p[highest].tat - p[highest].bt;
}
```



```

sumtat += p[highest].tat;
sumwt += p[highest].wt;
sumrt += p[highest].rt;

avgtat = (float) sumtat / n;
avgwt = (float) sumwt / n;
avgrt = (float) sumrt / n;

printf("\n P | AT | BT | PRI | CT | TAT | WT | RT\n");
printf("\n TAT | WT | RT | n");

for(i=0; i<n; i++) {
    printf("\n P: %d | %d | %d | %d | %d | %d | %d | %d",
           p[i].pid, p[i].at, p[i].bt, p[i].pr,
           p[i].ct, p[i].tat, p[i].wt, p[i].rt);
}

printf("\n Average TAT: %.2f", avgtat);
printf("\n Average WT: %.2f", avgwt);
printf("\n Average RT: %.2f", avgrt);
return 0;

```

Enter number of process: 3
 Enter AT, BT and Priority for P1: 1 5 3
 Enter AT, BT and Priority for P2: 4 2 1
 Enter AT, BT and Priority for P3: 6 3 1

P	AT	BT	PRI	CT	TAT	WT	RT
P1	1	5	3	10	9	4	2
P2	4	2	1	6	2	0	0
P3	6	3	2	3	3	0	0

Average TAT: 4.67
 Average WT: 1.33

```

#include <stdio.h>
struct Process {
    int pid, at, bt, ct, tat, wt, rt;
};

int main() {
    int n, i, j;
    struct Process p[20];
    int queue[100];
    int visited[20] = {0};
    int front = 0, rear = 0;
    int time = 0, completed = 0;
    int order[20], time mark[20], k = 0;
    float avgwt = 0, avgtat = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for(i=0; i<n; i++) {
        p[i].pid = i+1;
        printf("\n Process %d", i+1);
        printf("\n Arrival Time: ");
        scanf("%d", &p[i].at);
        printf("\n Burst Time: ");
        scanf("%d", &p[i].bt);
        p[i].rt = p[i].bt;
    }
    for(i=0; i<n-1; i++) {
        for(j=i+1; j<n; j++) {
            if(p[i].at > p[j].at ||
               (p[i].at == p[j].at & p[i].pr < p[j].pr)) {
                struct Process temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }

```



```

printf("\n Enter time Quantum. ");
scanf("%d", &tq);
time mark [0] = 0;
while (completed < n) {
    for (int i = 0; i < n; i++) {
        if (p[i].at <= time && visited[i] == 0) {
            queue[rear++] = i;
            visited[i] = 1;
        }
    }
    if (front == rear) {
        time++;
        continue;
    }
    int idx = queue[front++];
    order[k] = p[idx].pid;
    if (p[idx].rt > tq) {
        p[idx].rt = tq;
        time = tq;
    }
    else {
        time = p[idx].rt;
        p[idx].rt = 0;
        p[idx].ct = time;
        p[idx].tat = p[idx].ct - p[idx].at;
        p[idx].wt = p[idx].tat - p[idx].bt;
        avg_wt += p[idx].wt;
        completed++;
    }
    k++;
    time mark[k] = time;
    for (int i = 0; i < n; i++) {

```

Date _____
Page _____

```

if (p[i].at <= time && visited[i] == 0) {
    queue[rear++] = i;
    visited[i] = 1;
}
}
if (p[idx].rt > 0) queue[rear++] = idx;
}

printf("\n - Round Robin Schedul - ");
printf("\n quant chait: %d", tq);

for (int i = 0; i < k; i++) printf("%d ", order[i]);
printf("\n");

for (int i = 0; i < k; i++) printf("%d ", time mark[i]);
printf("\n");

printf("Process | TAT | BT | CT | TT | WT");

for (int i = 0; i < k; i++) {
    printf("P%d TAT: %d | BT: %d | CT: %d | TT: %d | WT: %d",
        p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);
    printf("\n");
}

printf("\n Average TAT = %d.2f", avgtat / n);
printf("\n Average WT = %d.2f", avgwt / n);

return 0;
}

```


Q1

Enter number of processes: 4.

Process 1

Arrival Time: 0

Burst Time: 5

Process 2

Arrival Time: 1

Burst Time: 7

Process 3

Arrival Time: 3

Burst Time: 4

Process 4

Arrival Time: 5

Burst Time: 5

Enter time quantum: 3

--- Round Robin Scheduling ---

Gantt Chart

P1 | P2 | P3 | P1 | P4 | P2 | P3 | P4 | P2 |
0 3 6 9 11 14 17 18 21 23

Process	AT	BT	CT	TAT	WT	RT
P1	0	5	11	14	9	0
P2	1	7	22	21	14	2
P3	3	4	18	15	11	2
P4	5	6	21	16	10	4

Average TAT = 15.25

Average WT = 10.25

For QT = 2; Avg TAT = 15.25
Avg WT = 9.75

Conclusion:
Avg TAT with RR ↓
Behaves like FCFS
content switching

#include <stdio.h>

struct Process {

int id;

int at;

int bt, ct, wt, tat, type, done; 3;

int main() {

int n;

printf("Enter no. of processes: ");

scanf("%d", &n);

struct Process p[n];

for(int i=0; i<n; i++) {

p[i].id = i+1;

p[i].done = 0;

printf("Enter Process %d\n", i+1);

printf("Enter AT: ");

scanf("%d", &p[i].at);

printf("Enter BT: ");

scanf("%d", &p[i].bt);

printf("Enter type (0: System, 1: User): ");

scanf("%d", &p[i].type);

}

int completed = 0, time = 0;

while (completed < n) {

int id = -1;


```

for (int priority=0; priority<1; priority++)
    for (int i=0; i<n; i++) {
        if (!p[i].done && p[i].at < time &&
            p[i].type == priority) {
            if (idn == -1 || p[i].at < p[idn].at)
                idn = i;
        }
    }

```

```

if (idn != -1) break;
if (idn != -1) {
    time += p[idn].bt;
    p[idn].ct = time;
    p[idn].tat = p[idn].ct - p[idn].at;
    p[idn].wt = p[idn].tat - p[idn].bt;
    p[idn].done = 1;
    cout << "Completed by " << idn << endl;
} else {
    time++;
}

```

```

float totalCOT = 0; totalTAT = 0;
printf("ID\tTAT\tAT\tBT\tCT\tWT\tTAT\n");

```

```

for (int i=0; i<n; i++) {
    printf("ID\tTAT\tAT\tBT\tCT\tWT\tTAT\n");
    printf("%d\t", p[i].id, p[i].at, p[i].bt, p[i].ct, p[i].wt, p[i].tat);
    totalWT += p[i].wt;
    totalTAT += p[i].tat;
}

```

```

printf("\n Avg Waiting Time: %.2f", totalWT/n);
printf("\n Avg Turnaround Time: %.2f", totalTAT/n);
return 0;
}

```

Q/p Enter number of processes: 4

Process 1

Arrival Time: 0

Burst Time: 5

Type (0=system, 1=user): 0

Process 2

Arrival Time: 2

Burst Time: 3

Type: 1

Process 3

Arrival Time: 1

Burst Time: 4

Type: 0

Process 4

Arrival Time: 3

Burst Time: 2

Type: 1

ID	Type	AT	BT	CT	WT	TAT
1	System	0	5	5	0	5
2	User	2	3	12	7	10
3	System	1	4	9	4	8
4	User	3	2	14	9	11

Avg waiting Time: 5.00

Avg turnaround Time: 8.50

Week-4

Rate Monotonic Scheduling

```
#include <stdio.h>
struct Process {
    int id, period, burst, remaining;
};

void rms(struct Process p[1], int n, int time_limit) {
    int time = 0;
    while (time < time_limit) {
        int highest = -1;

        for (int i = 0; i < n; i++) {
            if (p[i].remaining > 0) {
                if (highest == -1 || p[i].period < p[highest].period) {
                    highest = i;
                }
            }
        }

        if (highest != -1) {
            printf("Time %d -> P%d\n", time, p[highest].id);
            p[highest].remaining--;
        } else {
            printf("Time %d -> Idle\n", time);
        }
        time++;
    }
}

int main() {
    int n, time_limit;
    printf("Enter number of processes: ");
    scanf("%d", &n);
```

```
for (int i = 0; i < n; i++) {
    p[i].id = i + 1;
    printf("Enter period & burst time for P%d: ", p[i].id);
    scanf("%d %d", &p[i].period, &p[i].burst);
    p[i].remaining = p[i].burst;
}

printf("Enter total simulation time: ");
scanf("%d", &time_limit);
rms(p, n, time_limit);
return 0;
}
```

Op

Enter number of processes: 2
 Enter period & burst time for P1
 5 8
 Enter period & burst time for P2
 7 10
 Enter total simulation time: 15
 Time 0 -> P1
 Time 1 -> P1
 Time 2 -> P1
 Time 3 -> P1
 Time 4 -> P1
 Time 5 -> P1
 Time 6 -> P1
 Time 7 -> P1
 Time 8 -> P2
 Time 9 -> P2
 Time 10 -> P2
 Time 11 -> P2
 Time 12 -> P2
 Time 13 -> P2
 Time 14 -> P2

~~Week 6~~ Earliest Deadline First
REM: Rate Monotonic Scheduling

```
#include <stdio.h>
struct Process {
    int id, deadline, burst, remaining;
};

void edf(struct Process p[], int n, int time, int limit) {
    int time = 0;
    while (time < time + limit) {
        int earliest = -1;
        for (int i = 0; i < n; i++) {
            if (p[i].remaining > 0) {
                if (earliest == -1 || p[i].deadline < p[earliest].deadline) {
                    earliest = i;
                }
            }
        }
        if (earliest != -1) {
            printf("Time %d -> P%d\n", time, p[earliest].id);
            p[earliest].remaining--;
        } else {
            printf("Time %d -> Idle\n", time);
        }
        time++;
    }
}

int main() {
    int n, time, limit;
```

print f ("Enter no. of processes: ");
scanf ("%d", &n);
struct Process p[n];
for (int i = 0; i < n; i++) {
 p[i].id = i+1;
 printf ("Enter deadline & burst time for P%d: ", p[i].id);
 scanf ("%d %d", &p[i].deadline, &p[i].burst);
 p[i].remaining = p[i].burst;
}
print f ("Enter total simulation time: ");
scanf ("%d", &time limit);
edf (p, n, time, limit);
if (time == 0) {
 printf ("No process found\n");
} else {
 printf ("Enter deadline & burst time for P1: 4 1\n");
 printf ("Enter deadline & burst time for P2: 5 2\n");
 printf ("Enter deadline & burst time for P3: 20 3\n");
 printf ("Enter total simulation time: 10\n");
 printf ("Time 0 -> P1\n");
 printf ("Time 1 -> P2\n");
 printf ("Time 2 -> P2\n");
 printf ("Time 3 -> P3\n");
 printf ("Time 4 -> P3\n");
 printf ("Time 5 -> P3\n");
 printf ("Time 6 -> Idle\n");
 printf ("Time 7 -> Idle\n");
 printf ("Time 8 -> Idle\n");
 printf ("Time 9 -> Idle\n");
 printf ("Time 10 -> Idle\n");
}

Proportional Share Scheduling

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

struct Process {
    int id, tickets;
};

void lottery(struct Process p[], int n,
            int time_slices) {
    int total_tickets = 0;

    for (int i = 0; i < n; i++) {
        total_tickets += p[i].tickets;
    }

    srand(time(0));

    for (int t = 0; t < time_slices; t++) {
        int winner = rand() % total_tickets;
        int sum = 0;

        for (int i = 0; i < n; i++) {
            sum += p[i].tickets;
            if (winner < sum) {
                printf("Time %d -> P %d\n", t, p[i].id);
                sum -= p[i].tickets;
                break;
            }
        }
    }
}

int main() {
    int n, time_slices;
    printf("Enter no. of processes: ");
```

```
scanf("%d", &n);

struct Process p[n];

for (int i = 0; i < n; i++) {
    p[i].id = i + 1;

    printf("Enter tickets for P %d: ", p[i].id);
    scanf("%d", &p[i].tickets);
}

printf("Enter no. of time slices: ");
scanf("%d", &time_slices);
lottery(p, n, time_slices);

return 0;
}
```

Enter no. of processes: 3
 Enter tickets for P1: 10
 Enter tickets for P2: 20
 Enter tickets for P3: 30
 Time 0 -> P3
 Time 1 -> P2
 Time 2 -> P3
 Time 3 -> P2
 Time 4 -> P3
 Time 5 -> P3
 Time 6 -> P2
 Time 7 -> P1
 Time 8 -> P3
 Time 9 -> P1

Lab 5 Bounded Buffer

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define SIZE 5

int buffer[SIZE];
int in = 0, out = 0;
int p_count, c_count;
sem_t mutex, empty, full;

void* producer(void* arg) {
    for (int i = 0; i < p_count; i++) {
        sem_wait(&empty);
        sem_wait(&mutex);
        buffer[in] = i;
        printf("Produced: %d at buffer[%d]\n", i, in);
        in = (in + 1) % SIZE;
        sem_post(&mutex);
        sem_post(&full);
    }
    return NULL;
}

void* consumer(void* arg) {
    for (int i = 0; i < c_count; i++) {
        sem_wait(&full);
        sem_wait(&mutex);
```

```
int item = buffer[out];
printf("Consumed: %d from Buffer\n", item, out);
```

```
out = (out + 1) % SIZE;
```

```
sem_post(&mutex);
sem_post(&empty);
}
```

```
return NULL;
```

```
}
int main() {
    pthread_t p, c;
```

```
printf("Enter no. of items to produce: ");
scanf("%d", &p_count);
```

```
printf("Enter no. of items to consume: ");
scanf("%d", &c_count);
```

```
sem_init(&mutex, 0, 1);
sem_init(&empty, 0, SIZE);
sem_init(&full, 0, 0);
```

```
pthread_create(&p, NULL, producer, NULL);
pthread_create(&c, NULL, consumer, NULL);
```

```
pthread_join(p, NULL);
pthread_join(c, NULL);
return 0;
```


Qp Enter number of items to produce: 3
Enter number of items to consume: 2

produced: 0 at buffer[0]
produced: 1 at buffer[1]
produced: 2 at buffer[2]
consumed: 0 from buffer[0]
consumed: 1 from buffer[1]

Trace: In=0, out=0
e=5 full=0

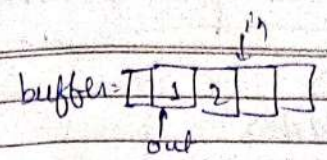
~~buffer = [0, 1, 2, 3]~~

Step 1:- produce 0
e → 5 → 4, store at buffer[0]
In=1
full=0 → 1, unlock mutex
buffer: [0] [] [] []

Step 2:- produce 1
empty=3
store 1 at buffer[1]
In=2
full=2, unlock buffer [0] [1] [] []

Step 3:- produce 2
e=2
store 2 at buffer[2]
In=3
full=3 buffer [0] [1] [2] []

Step 4:- Consume 0
full=2
take from buffer[0] out=1 empty=3



Step 5:- Consume 1
full → 1
out = 2
e = 4
take from buffer[1]
Buffer [] [2] [] []

Dining Philosophers Problem

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define MAX 10

int n;
sem_t forks[MAX];

void *philosopher(void *num) {
    int id = *(int *) num;
    printf("Philosopher %d is thinking.\n", id);
    sleep(1);

    sem_wait(&forks[(id+1) % n]);

    printf("Philosopher %d picked up right fork %d.\n", id, (id+1) % n);

    sem_wait(&forks[id]);
    printf("Philosopher %d picked up left fork %d.\n", id, id);

    printf("Philosopher %d is eating.\n", id);
    sleep(1);

    sem_post(&forks[id]);
    sem_post(&forks[(id+1) % n]);

    printf("Philosopher %d put down forks %d & %d.\n", id, id, (id+1) % n);
    return NULL;
}
```

```
int main() {
    pthread_t p[MAX];
    int ids[MAX];

    printf("Enter number of philosophers:");
    scanf("%d", &n);

    for (int i=0; i<n; i++)
        sem_init(&forks[i], 0, 1);

    for (int i=0; i<n; i++)
        ids[i] = i;
    pthread_create(&p[i], NULL, philosopher, &ids[i]);
}

for (int i=0; i<n; i++)
    pthread_join(p[i], NULL);

return 0;
}
```

O/P Enter number of philosophers: 3
 Philosopher 0 is thinking.
 " 2 is "
 " 1 " "
 0 0 picked up right fork 1.
 " 0 " " left " 0.
 " 0 is eating.
 " 1 picked up right fork 2.
 " 0 put down forks 0 & 1.

Philosopher 0 is thinking.

" 1 picked up left fork 1.

" 1 is eating.

" 2 picked up right fork 0.

" 1 put down forks 1 & 2.

" 2 picked up left fork 2.

" 2 is eating.

" 1 is thinking.

" 2 put down forks 2 & 0.

" 2 is thinking.

Trace

Initial : 3 philosophers (0, 1, 2)
forks : 0, 1, 2 (all free)

Step 1 P0 P1 P2 → thinking

Step 2 P0 eats, P0 picks right 1
P0 picks left 0
P0 eats
Forks use: 0 & 1

Step 3 P1 waits then eats

P1 picks right fork 2
waits

P0 releases 0 & 1 forks

P1 picks left 1 fork

P1 eats

Forks use: 1 & 2

Step 4 P2 eats

Lab 6 2/5/26
Banker's Algo

#include <stdio.h>

#define MAX_P 10

#define MAX_R 10

int n, m;

int Allocation [MAX_P] [MAX_R];

int Max [MAX_P] [MAX_R];

int Need [MAX_P] [MAX_R];

int Available [MAX_R];

void Calculate Need () {

for (int i = 0; i < n; i++) {

for (int j = 0; j < m; j++) {

Need[i][j] = Max[i][j] - Allocation[i][j];

}

}

}

int SafetyAlgorithm () {

int Work [MAX_R];

int Finish [MAX_P] = {0};

int SafeSequence [MAX_P];

int count = 0;

for (int i = 0; i < m; i++) {

work[i] = Available[i];

}

while (count < n) {

int found = 0;

for (int i = 0; i < n; i++) {

if (Finish[i] == 0) {


```
init (Finish[i] := 0);
```

```
int possible = 1;
```

```
for (int j = 0; j < m; j++) {
```

```
    if (Need[i][j] > Work[j]) {  
        possible = 0;  
        break;  
    }
```

```
}
```

```
if (possible) {
```

```
    for (int j = 0; j < m; j++) {  
        work[j] += Allocation[i][j];  
    }
```

```
    SafeSequence[Count++]: i;  
    Finish[i] = 1;
```

```
    found = 1;
```

```
}
```

```
}
```

```
}
```

```
void resource_request() {
```

```
    int p;  
    int Request[100];
```

```
    printf("\nEnter process no. making request: ");  
    scanf("%d", &p);
```

```
printf("Enter request vector: \n");
```

```
for (int i = 0; i < m; i++) {  
    scanf("%d", &Request[i]);
```

```
}
```

```
for (int i = 0; i < m; i++) {  
    if (Request[i] > Need[p][i]) {  
        printf("\nError: Process exceeded  
        maximum claim. \n");
```

```
        return;
```

```
}
```

```
for (int i = 0; i < m; i++) {  
    if (Request[i] > AvailableNeed[p][i]) {  
        printf("\nError: Process resources  
        not available. Process must wait);
```

```
        return;
```

```
}
```

```
for (int i = 0; i < m; i++) {  
    Available[i] -= Request[i];  
    Allocation[p][i] += Request[i];  
    Need[p][i] -= Request[i];
```

```
}
```

```
printf("\nChecking system safety  
after allocation. \n");
```

```
if (SafetyAlgo()) {  
    printf("\nRequest can be Granted \n");
```

```
}
```



```

else {
    for (int i = 0; i < m; i++) {
        Available[i] += Request[i];
        Allocation[P][i] -= Request[i];
        Need[P][i] += Request[i];
    }
    printf("Request Cannot be granted.\nRolling back");
}
}

int main() {
    printf("Enter number of processes");
    scanf("%d", &n);
    printf("Enter resource type");
    scanf("%d", &m);
    printf("Enter Allocation Matrix\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &Max[i][j]);
        }
    }
    printf("Enter Maximum Matrix\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &Max[i][j]);
        }
    }
    printf("Enter Av. Resources: ");
    for (int i = 0; i < m; i++) {
        scanf("%d", &Available[i]);
    }
}

```

Date _____
Page _____

```

Calculate Need();
printf("Need Matrix");
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        printf("%d", Need[i][j]);
    }
    printf("\n");
}
printf("Initial Safety Check");

safetyAlgorithm();
resourceRequest();

return 0;
}

Enter number of processes: 5
Enter " resource types: 3
Enter Allocation Matrix;
0 1 0
2 0 0
3 0 4
2 1 1
0 0 2

Enter Maximum Matrix;
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Available Resources;
3 3 2

Need Matrix
7 4 3
1 2 2
6 0 0
0 0 1
4 3 1

Initial Safety Check:
System is in SAFE STATE
Safe Sequence:
P1 → P3 → P4 → P0 → P2

```


Deadlock Detection

```
#include <stdio.h>
#define MAX_P 10
#define MAX_R 10

int main() {
    int n, m;
    int Allocation[MAX_P][MAX_R];
    int Request[MAX_P][MAX_R];
    int Available[MAX_R];

    int work[MAX_R];
    int finish[MAX_P];

    printf("Enter no. of processes: ");
    scanf("%d", &n);

    printf("Enter no. of resource types: ");
    scanf("%d", &m);

    printf("\nEnter Allocation Matrix:\n");

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &Allocation[i][j]);
        }
    }

    printf("Enter Request Matrix:\n");

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &Request[i][j]);
        }
    }
}
```

```
printf("\nEnter Available Resources:\n");

for (int i = 0; i < m; i++) {
    scanf("%d", &Available[i]);
}

for (int i = 0; i < m; i++) {
    work[i] = Available[i];
}

for (int i = 0; i < n; i++) {
    int allZero = 1;

    for (int j = 0; j < m; j++) {
        if (Allocation[i][j] != 0) {
            allZero = 0;
            break;
        }
    }

    if (allZero) {
        finish[i] = 1;
    } else {
        finish[i] = 0;
    }
}

int found;
do {
    found = 0;
    for (int i = 0; i < n; i++) {
        if (finish[i] == 0) {
            int possible = 1;

```



```

if (possible) {
    for (int j = 0; j < m; j++) {
        work[j] += Allocation[i][j];
        finish[i] = 1;
        found = 1;
    }
}

```

```

printf("\n Process P%d has finished", i);
}
}

```

```

while (found);
int deadlock = 0;
printf("\n\n Deadlocked Processes:");

```

```

for (int i = 0; i < n; i++) {
    if (finish[i] == 0) {
        printf("P%d, ", i);
        deadlock = 1;
    }
}

```

```

if (!deadlock) printf("None");

```

```

if (deadlock)
    printf("System is in Deadlock");

```

```

else
    printf("System is not in deadlock");

```

```

return 0;
}

```

if Enter number of processes: 3
Enter number of resource types: 2

Enter Allocation Matrix

1	0
0	1
1	1

Enter Request Matrix

0	1
0	0
1	1

Enter Available Resources:

0	0
---	---

Deadlocked Processes: P0, P1, P2

System is in Deadlock state.

5/5/26

lab-7

Given 6 memory partitions of 300KB, 600KB, 250KB, 750KB, 125KB (in order). How would best fit, first fit, worst fit, algo place process of size 115KB, 500KB, 358KB, 200KB, 375KB (in order).

Sol:- First fit:-

115KB	500KB	200KB	358KB
-------	-------	-------	-------

+ 300KB - 600KB - 250KB - 750KB - 125KB

Process with size 375 can't be allocated with memory.

Best fit:-

500KB	200KB	358KB	115KB
-------	-------	-------	-------

300KB 600KB 250KB 750KB 125KB

process with size 375KB can't be allocated mem

Worst fit

500KB	200KB	115KB
-------	-------	-------

300 600 250 750 125

process with size 358KB & 375KB can't be allocated memory.

Write a program to execute the contiguous memory allocation techniques.

① Worst fit ② Best fit ③ First fit

```
#include <stdio.h>
int main() {
    int memory[50], process[50];
    int m, n;
    int i, j;
    printf("Enter number of memory blocks: ");
    scanf("%d", &m);
    printf("Enter sizes of %d memory block\n", m);
    for(i=0; i<m; i++) {
        scanf("%d", &memory[i]);
    }
    printf("Enter no. of processes: ");
    scanf("%d", &n);
    printf("Enter sizes of %d processes: \n");
    for(i=0; i<n; i++) {
        scanf("%d", &process[i]);
    }
    int firstFit[50];
    int tempMemory[50];
    for(i=0; i<m; i++) {
        tempMemory[i] = memory[i];
    }
    for(i=0; i<n; i++) {
        firstFit[i] = -1;
        for(j=0; j<m; j++) {

```



```

if (tempMemory[j] > process[i])
{
    firstFit[i] = j;
    tempMemory[j] -= process[i];
    break;
}
}
}

```

```

printf("First Fit);
printf("Process No | Process Size | Block No\n");

```

```

for (i=0; i<n; i++) {
    printf("%d | %d | %d\n", i+1,
        process[i]);
}

```

```

if (firstFit[i] != -1) printf("%d\n", firstFit[i]+1);
else printf("Not Allocated");
}

```

```

int bestFit[50];
int tempMemory2[50];

```

```

for (i=0; i<m; i++) {
    tempMemory2[i] = Memory[i];
}

```

```

for (i=0; i<n; i++) {
    int bestIndex = -1;
}

```

```

for (j=0; j<m; j++) {
    if (tempMemory2[j] > process[i])

```

```

{
    if (bestIndex == -1 || tempMemory2[j] < tempMemory2[bestIndex])
    {
        bestIndex = j;
    }
}
}

```

```

if (bestIndex != -1) {
    bestFit[i] = bestIndex;
    tempMemory2[bestIndex] -= process[i];
}
else {
    bestFit[i] = -1;
}
}

```

```

printf("Best Fit);
printf("Process No | Process Size | Block No\n");
for (i=0; i<n; i++) {
    printf("%d | %d | %d\n", i+1, process[i],
        bestFit[i]);
}

```

```

if (bestFit[i] != -1) printf("%d\n", bestFit[i]+1);
else printf("Not Allowed");
}

```

```

int worstFit[50];
int tempMemory3[50];
for (i=0; i<m; i++) {
    tempMemory3[i] = memory[i];
}

```

```

for (i=0; i<n; i++) {
    int worstIndex = -1;
    for (j=0; j<m; j++) {

```



```

if (tempMemory3[j] >= process[i]) {
    if (worstIndex == -1 || tempMemory3[j] >
        tempMemory3[worstIndex])
    }

```

```

    worstIndex = j;
}

```

```

if (worstIndex != -1) {
    worstFit[i] = worstIndex;
    tempMemory3[worstIndex] -= process[i];
} else {
    worstFit[i] = -1;
}

```

```

printf("Worst Fit");
printf("Process No | Process Size | Block No\n");
for (i = 0; i < n; i++) {
    printf("%d | %d | %d", i+1, process[i], worstFit[i]);
    if (worstFit[i] != -1)
        printf("\n");
    else
        printf("Not Allocated\n");
}
return 0;

```

o/p: Enter number of process block: 5
 Enter sizes of 5 memory blocks:
 100 500 200 200 600
 Enter number of processes: 4
 Enter sizes of 4 processes:
 212 417 112 426

First Fit

Process No	Process Size	Block No
1	212	2
2	417	5
3	112	2
4	426	Not Allocated

Best Fit

Process No	Process Size	Block No
1	212	4
2	417	2
3	112	3
4	426	5

Worst Fit

Process No	Process Size	Block No
1	212	5
2	417	2
3	112	5
4	426	Not Allocated

512
 112

Page- Replacement Algorithm:-

FIFO:-

#include <stdio.h>

int main() {

int n, i, j, k = 0, frames;

printf("Enter no. of pages:");

scanf("%d", &n);

int pages[n];

printf("Enter page reference string:");

for (int i = 0; i < n; i++) {

scanf("%d", &pages[i]);

}

printf("Enter no. of frames:");

scanf("%d", &frames);

int frame[frames];

for (int i = 0; i < frames; i++) {

frame[i] = -1;

}

int fault = 0, index = 0;

for (int i = 0; i < n; i++) {

int found = 0;

for (int j = 0; j < frames; j++) {

if (frame[j] == pages[i]) {

found = 1;

break;

}

if (!found) {

frame[index] = pages[i];

index = (index + 1) % frames;

fault;

}

for (int j = 0; j < frames; j++) {

if (frame[j] != -1)

printf("%d", frame[j]);

else

printf(" - ");

}

printf("\n\nTotal Page Faults = %d\n", fault);

return 0;

}

Output:-

Enter number of pages: 4

Enter page reference string: 1

0

2

0

Enter number of frames: 3

1

1

1

0

2

LRU :-

```
#include <stdio.h>
int main() {
    int pages[50], frames[10];
    int n, f, i, j, k;
    int pageFaults = 0;

    printf("Enter no. of pages: ");
    scanf("%d", &n);

    printf("Enter page reference string: ");
    for (int i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    printf("Enter no. of frames: ");
    scanf("%d", &f);

    for (i = 0; i < f; i++)
        frames[i] = -1;

    for (i = 0; i < n; i++) {
        int found = 0;

        for (j = 0; j < f; j++) {
            if (frames[j] == pages[i]) {
                found = 1;
                break;
            }
        }

        if (!found) {
            // Page fault
            // Find the least recent page to replace
            int leastRecent = -1;
            for (k = 0; k < f; k++) {
                if (frames[k] != pages[i]) {
                    if (leastRecent == -1 ||
                        frames[leastRecent] < frames[k])
                        leastRecent = k;
                }
            }

            // Replace the least recent page
            frames[leastRecent] = pages[i];
            pageFaults++;
        }

        printf("Page %d -> ", pages[i]);
        for (j = 0; j < f; j++)
            printf("%d ", frames[j]);
        printf("\n");
    }

    printf("Total Page Faults = %d\n", pageFaults);
    return 0;
}
```

Date _____
Page _____
PIYUSH

```
int leastRecent = -1;
for (j = 0; j < f; j++) {
    if (frames[j] == pages[i]) {
        leastRecent = j;
        break;
    }
}

if (leastRecent == -1) {
    // Page fault
    // Find the least recent page to replace
    int leastRecent = -1;
    for (k = 0; k < f; k++) {
        if (frames[k] != pages[i]) {
            if (leastRecent == -1 ||
                frames[leastRecent] < frames[k])
                leastRecent = k;
        }
    }

    // Replace the least recent page
    frames[leastRecent] = pages[i];
    pageFaults++;
}

printf("Page %d -> ", pages[i]);
for (j = 0; j < f; j++)
    printf("%d ", frames[j]);
printf("\n");

printf("Total Page Faults = %d\n", pageFaults);
return 0;
}
```

Output :- Enter no. of pages: 4
 Enter page reference string: 2 3 1 3
 Enter no. of frames: 2
 Page 2 -> 2 -1
 Page 3 -> 2 3
 Page 1 -> 1 3
 Page 3 -> 1 3
 Total Page Faults = 3

Optimal Page Fed:-

```
#include <stdio.h>
int main () {
    int pages [10], frames [10];
    int n, f, i, j, k;
    int page faults = 0, found;

    printf ("Enter no. of pages: ");
    scanf ("%d", &n);

    printf ("Enter page reference string: ");
    for (i = 0; i < n; i++)
        scanf ("%d", &pages[i]);

    printf ("Enter no. of frames: ");
    scanf ("%d", &f);

    for (i = 0; i < f; i++)
        frames[i] = -1;

    for (i = 0; i < n; i++) {
        found = 0;
        for (j = 0; j < f; j++) {
            if (frames[j] == pages[i]) {
                found = 1;
                break;
            }
        }
        if (!found) {
            int replace = -1;
            for (j = 0; j < f; j++) {
                if (frames[j] == -1)
                    replace = j;
                break;
            }
            if (replace == -1)
                break;
            frames[replace] = pages[i];
            page faults++;
        }
    }

    printf ("\n Total Page Faults = %d", page faults);
    return 0;
}
```

```
if (next use > farthest) {
    farthest = next use;
    replace = j;
}

frames[replace] = pages[i];
page faults++;

printf ("Page %d →", pages[i]);
for (j = 0; j < f; j++)
    printf ("%d", frames[j]);
printf ("\n");

printf ("\n Total Page Faults = %d", page faults);
return 0;
}
```

Output

```
Enter no. of pages: 4
Enter page ref string: 1 2 3 1
Enter no. of frames: 2
Page 1 → 1 -1
Page 2 → 1 2
Page 3 → 1 3
Page 1 → 1 3
Total Page Faults = 3.
```